Laboratorio 6 - K Nearest Neighbors (KNN)

Joel Jaquez, Luis Gonzalez, Fabian Morales

2026-04-10

Preprocesamiento de datos para KNN de Regresión

```r
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(readr)
library(tidyr)
library(caret)
```

```
## Loading required package: ggplot2

## Loading required package: lattice
```

```r
library(ggplot2)
library(class)
```

```r
load("datos/listings.RData")
```

```r
listings_clean_comp <- listings %>%
  mutate(price = parse_number(price)) %>%
  select(-c(listing_url, scrape_id, picture_url, host_url,
            host_thumbnail_url, host_picture_url, description,
            neighborhood_overview, host_about, amenities))

# Seed y split 80/20
set.seed(123)
```

```r
total_filas  <- nrow(listings_clean_comp)
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)
train_comp   <- listings_clean_comp[ indice_train, ]
test_comp    <- listings_clean_comp[-indice_train, ]

# filtros y variables
# price, accommodates, bedrooms, bathrooms,
# number_of_reviews, review_scores_rating
train_modelo <- train_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

test_modelo <- test_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

cat("Filas entrenamiento:", nrow(train_modelo), "\n")
```

```
## Filas entrenamiento: 48442
```

```r
cat("Filas prueba:       ", nrow(test_modelo),  "\n")
```

```
## Filas prueba:        12173
```

Actividad 1 - Modelo de Regresión usando KNN

```r
# k inicial = raíz cuadrada de n (criterio estándar para KNN)
k_vecinos <- round(sqrt(nrow(train_modelo)), 0)
cat("Valor de k utilizado:", k_vecinos, "\n")
```

```
## Valor de k utilizado: 220
```

```r
parametros <- expand.grid(k = k_vecinos)

modelo_knn <- train(
  price ~ .,
  data       = train_modelo,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = parametros
)
```

```r
print(modelo_knn)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 48442, 48442, 48442, 48442, 48442, 48442, ...
## Resampling results:
##
##   RMSE      Rsquared   MAE
##   130.6013  0.4054889  91.00289
##
## Tuning parameter 'k' was held constant at a value of 220
```

Para el modelo de regresión con KNN se utilizó como punto de partida el criterio estándar de selección de k, tomando la raíz cuadrada del número de observaciones del conjunto de entrenamiento, lo que resultó en k=220 vecinos. El modelo fue entrenado con las 48,442 observaciones disponibles y las 5 variables predictoras: accommodates, bedrooms, bathrooms, number_of_reviews y review_scores_rating, aplicando normalización mediante centrado y escalado para garantizar que ninguna variable domine el cálculo de distancias por tener una escala mayor. Los resultados muestran un RMSE de 130.78, un R^2 de 0.4069 y un MAE de 91.13, lo que indica que en promedio el modelo se equivoca en aproximadamente 91 por noche al predecir el precio de una propiedad. El R^2 de 0.41 significa que el modelo logra explicar el 41% de la variabilidad del precio, un resultado moderado que es consistente con los modelos anteriores y refleja que el precio de una propiedad en Airbnb depende también de factores que no están capturados en estas variables numéricas, como el tipo de propiedad, la ubicación específica o las amenidades

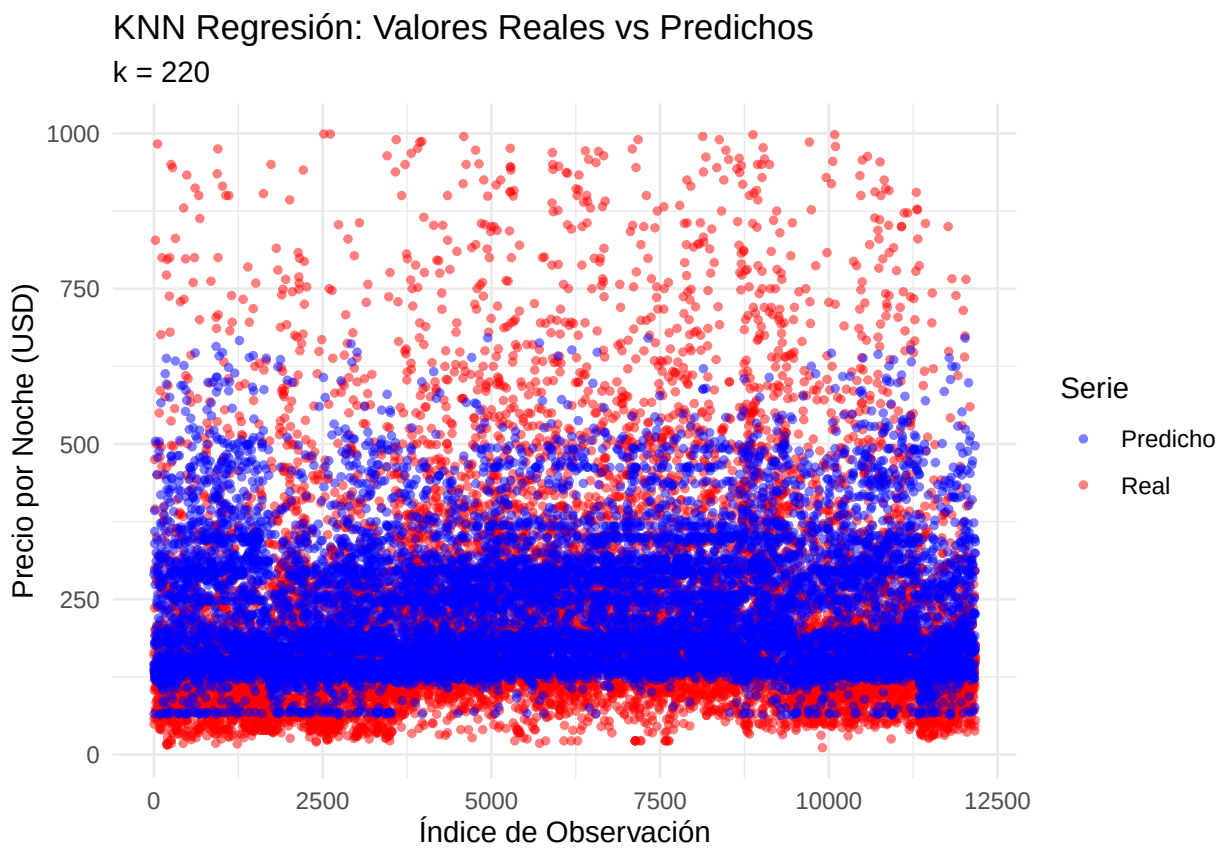
```r
pred_knn <- predict(modelo_knn, newdata = test_modelo)

cat("RMSE:", round(RMSE(pred_knn, test_modelo$price), 2), "\n")
```

```
## RMSE: 130.89
```

El RMSE obtenido al predecir sobre el conjunto de prueba fue de 130.89, valor prácticamente idéntico al RMSE de entrenamiento (130.78), lo cual es una señal positiva ya que indica que el modelo generaliza bien y no está sobreajustado a los datos con los que fue entrenado. En otras palabras, el modelo se comporta de manera consistente tanto con datos conocidos como con datos nuevos, que es exactamente lo que se busca en un modelo predictivo confiable.

```r
df_resultados <- data.frame(
  Indice   = 1:length(test_modelo$price),
  Real     = test_modelo$price,
  Predicho = pred_knn
)

ggplot(df_resultados) +
  geom_point(aes(x = Indice, y = Real,     color = "Real"),
             size = 1, alpha = 0.5) +
  geom_point(aes(x = Indice, y = Predicho, color = "Predicho"),
```

```
             size = 1, alpha = 0.5) +
  scale_color_manual(values = c("Real" = "red", "Predicho" = "blue")) +
  labs(title    = "KNN Regresión: Valores Reales vs Predichos",
       subtitle = paste0("k = ", k_vecinos),
       x = "Índice de Observación",
       y = "Precio por Noche (USD)",
       color = "Serie") +
  theme_minimal()
```



El gráfico muestra la comparación entre los precios reales (rojo) y los predichos por el modelo KNN (azul) para las 12,173 observaciones del conjunto de prueba. Lo más notable es que los puntos azules forman una banda compacta concentrada principalmente entre 100 y 400, lo que revela una limitación característica de KNN con k grande: al promediar 220 vecinos, el modelo tiende a predecir valores cercanos a la media del precio y pierde capacidad para capturar los extremos. Esto se hace evidente al observar que los puntos rojos se dispersan ampliamente hasta los 1,000 por noche, mientras que las predicciones azules raramente superan los 500. En el rango de precios bajos y medios el modelo se comporta razonablemente bien, ya que ambas nubes de puntos se solapan con frecuencia, pero en el segmento de propiedades caras el modelo subestima consistentemente, lo cual es coherente con el RMSE de $130.89 obtenido anteriormente.

Actividad 2 - Análisis de Resultados

```
rmse_knn <- RMSE(pred_knn, test_modelo$price)
mae_knn  <- MAE(pred_knn,  test_modelo$price)
```

```r
r2_knn   <- cor(pred_knn,  test_modelo$price)^2

cat("===== MÉTRICAS - KNN Regresión =====\n")
```

```
## ===== MÉTRICAS - KNN Regresión =====
```

```r
cat(sprintf("RMSE : $%.2f\n", rmse_knn))
```

```
## RMSE : $130.89
```

```r
cat(sprintf("MAE  : $%.2f\n", mae_knn))
```

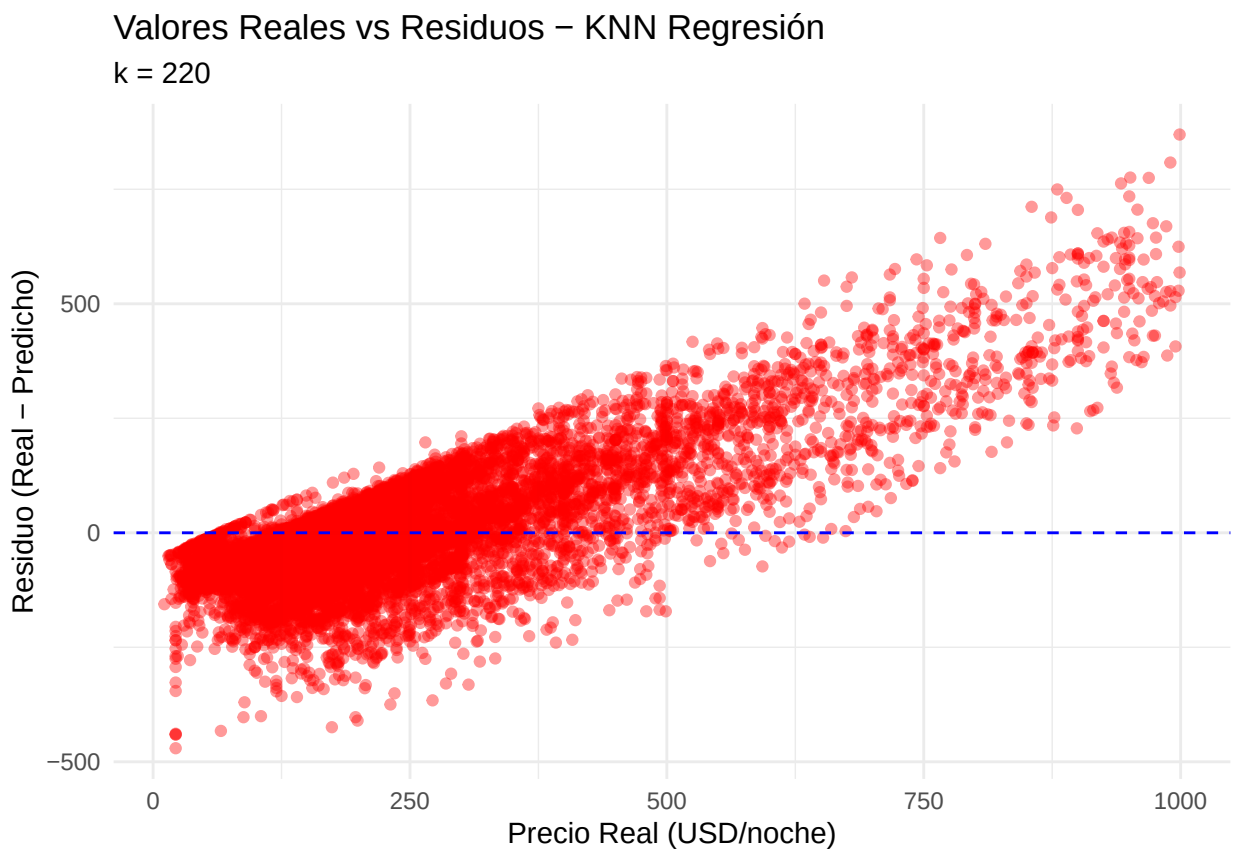
```
## MAE  : $90.81
```

```r
cat(sprintf("R²   :  %.4f\n", r2_knn))
```

```
## R²   :  0.4141
```

Las tres métricas obtenidas permiten evaluar con precisión el desempeño del modelo. El RMSE de 130.89 indica que, en promedio, las predicciones del modelo se desvían aproximadamente 131 del precio real por noche, siendo esta métrica especialmente sensible a los errores grandes, como los que ocurren con propiedades de precio muy alto. El MAE de 90.81 complementa este análisis al mostrar que el error típico sin penalizar los extremos es de casi 91, lo que significa que para propiedades de precio medio el modelo tiene un desempeño más aceptable de lo que sugiere el RMSE. Finalmente, el R^2 de 0.4141 indica que el modelo explica el 41.41% de la variabilidad del precio, dejando casi el 60% sin explicar. Esto no es sorprendente porque como se trabajó únicamente con 5 variables numéricas, cuando en realidad el precio de una propiedad en Airbnb está influenciado por muchos otros factores como la ubicación exacta, el tipo de propiedad o las amenidades disponibles. En general, el modelo tiene un desempeño moderado, comparable con los algoritmos evaluados en entregas anteriores, y su mayor debilidad está en la predicción de propiedades en los extremos del rango de precios.

```r
residuos <- test_modelo$price - pred_knn

df_residuos <- data.frame(
  Real    = test_modelo$price,
  Residuo = residuos
)

ggplot(df_residuos, aes(x = Real, y = Residuo)) +
  geom_point(color = "red", size = 1.5, alpha = 0.4) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "blue") +
  labs(title    = "Valores Reales vs Residuos - KNN Regresión",
       subtitle = paste0("k = ", k_vecinos),
       x = "Precio Real (USD/noche)",
       y = "Residuo (Real - Predicho)") +
  theme_minimal()
```



El gráfico de residuos revela un patrón muy claro y preocupante porque los errores no están distribuidos aleatoriamente alrededor de la línea de cero, sino que forman una banda diagonal que se abre progresivamente hacia la derecha. Esto significa que conforme el precio real de la propiedad aumenta, el modelo tiende a subestimar más y más, generando residuos positivos cada vez mayores. En propiedades con precios bajos, entre 0 y 250, los puntos se concentran cerca de la línea azul con errores relativamente pequeños y bastante negativos, lo que indica que en ese rango el modelo sobreestima ligeramente. A partir de los 500 por noche el patrón se invierte completamente y los residuos se vuelven fuertemente positivos, confirmando que el modelo no logra seguirle el ritmo a las propiedades más caras. Este comportamiento es una consecuencia directa de usar k=220 vecinos, ya que al promediar tantas observaciones el modelo queda anclado cerca de la media general del precio y pierde sensibilidad hacia los extremos. En conclusión, el modelo KNN funciona razonablemente bien para propiedades de precio medio pero falla sistemáticamente con el segmento premium, lo cual es una limitación importante para los objetivos de SmartStay Advisors.

Actividad 3 - Comparacion

```r
# Re-entrenar modelos anteriores con las MISMAS condiciones
# set.seed(123), 80/20, mismas 6 variables

# Regresión Lineal (Lab 3)
modelo_lineal <- lm(price ~ ., data = train_modelo)
pred_lineal   <- pmax(predict(modelo_lineal, newdata = test_modelo), 0)
rmse_lineal   <- RMSE(pred_lineal, test_modelo$price)
mae_lineal    <- MAE(pred_lineal,  test_modelo$price)
r2_lineal     <- cor(pred_lineal,  test_modelo$price)^2
```

```r
# Árbol de Regresión profundidad 10 (mejor modelo Lab 4)
library(rpart)
arbol_comp <- rpart(price ~ ., data = train_modelo, method = "anova",
                    control = rpart.control(maxdepth = 10))
pred_arbol <- predict(arbol_comp, newdata = test_modelo)
rmse_arbol <- RMSE(pred_arbol, test_modelo$price)
mae_arbol  <- MAE(pred_arbol,  test_modelo$price)
r2_arbol   <- cor(pred_arbol,  test_modelo$price)^2

# Naive Bayes (Lab 5)
library(e1071)
```

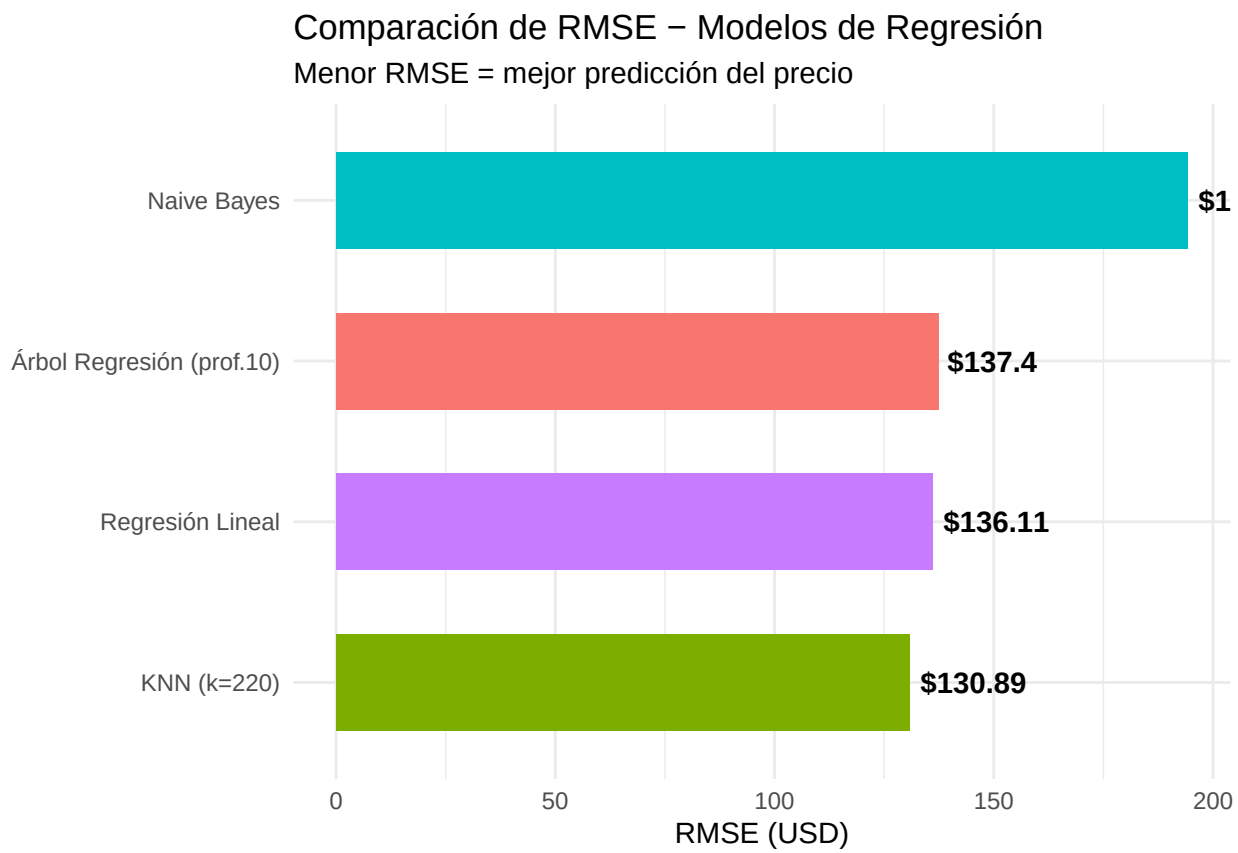
```
##
## Attaching package: 'e1071'

## The following object is masked from 'package:ggplot2':
##
##     element
```

```r
discretizar <- function(df) {
  df %>% mutate(
    accommodates_d = cut(accommodates, c(0,2,4,6,Inf),
                         labels=c("1-2","3-4","5-6","7+")),
    bedrooms_d     = cut(bedrooms,     c(0,1,2,3,Inf),
                         labels=c("1","2","3","4+")),
    bathrooms_d    = cut(bathrooms,    c(0,1,2,3,Inf),
                         labels=c("1","2","3","4+")),
    reviews_d      = cut(number_of_reviews, c(-1,0,10,50,Inf),
                         labels=c("0","1-10","11-50","50+")),
    rating_d       = cut(review_scores_rating, c(0,60,80,90,100),
                         labels=c("bajo","medio","alto","excelente"))
  )
}

train_nb <- discretizar(train_modelo)
test_nb  <- discretizar(test_modelo)
price_breaks <- quantile(train_nb$price, probs = seq(0,1,by=0.05))
train_nb$price_bin <- cut(train_nb$price, breaks=price_breaks,
                          include.lowest=TRUE, labels=FALSE)
bin_midpoints <- sapply(1:(length(price_breaks)-1), function(i)
  (price_breaks[i]+price_breaks[i+1])/2)
features_nb <- c("accommodates_d","bedrooms_d","bathrooms_d",
                 "reviews_d","rating_d")
nb_comp     <- naiveBayes(x=train_nb[,features_nb],
                          y=as.factor(train_nb$price_bin))
pred_bin_nb <- predict(nb_comp, newdata=test_nb[,features_nb])
pred_bin_num <- pmin(pmax(as.numeric(as.character(pred_bin_nb)),1),
                     length(bin_midpoints))
pred_nb      <- bin_midpoints[pred_bin_num]
rmse_nb      <- sqrt(mean((test_modelo$price - pred_nb)^2, na.rm=TRUE))
mae_nb       <- mean(abs(test_modelo$price - pred_nb), na.rm=TRUE)
r2_nb        <- cor(test_modelo$price, pred_nb, use="complete.obs")^2
```

```r
comparacion_reg <- data.frame(
  Modelo = c("Regresión Lineal",
             "Árbol Regresión (prof.10)",
             "Naive Bayes",
             "KNN (k=220)"),
  RMSE   = round(c(rmse_lineal, rmse_arbol, rmse_nb, rmse_knn), 2),
  MAE    = round(c(mae_lineal,  mae_arbol,  mae_nb,  mae_knn),  2),
  R2     = round(c(r2_lineal,   r2_arbol,   r2_nb,   r2_knn),   4)
)

print(comparacion_reg)
```

```
##                       Modelo   RMSE    MAE     R2
## 1          Regresión Lineal 136.11  94.95 0.3659
## 2 Árbol Regresión (prof.10) 137.40  95.50 0.3535
## 3               Naive Bayes 194.25 132.59 0.3428
## 4               KNN (k=220) 130.89  90.81 0.4141
```

```r
ggplot(comparacion_reg,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0("$", RMSE)),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  labs(title    = "Comparación de RMSE - Modelos de Regresión",
       subtitle = "Menor RMSE = mejor predicción del precio",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```



La comparación entre los cuatro modelos de regresión evaluados bajo las mismas condiciones muestra que KNN con k=220 es el algoritmo que mejor predice el precio por noche de las propiedades, obteniendo el menor RMSE de \$130.89, el menor MAE de \$90.81 y el mayor R^2 de 0.4141. Esto significa que KNN logra explicar el 41.41% de la variabilidad del precio, superando a todos los modelos anteriores. En segundo lugar queda la Regresión Lineal con un RMSE de \$136.11 y R^2 de 0.3659, un resultado competitivo considerando su simplicidad. El Árbol de Regresión con profundidad 10 queda en tercer lugar con RMSE de \$137.40 y R^2 de 0.3535, resultado que llama la atención ya que en el Lab 4 superaba a la Regresión Lineal cuando tenía acceso a todas las variables numéricas del dataset, pero al restringirse a las mismas 6 variables pierde esa ventaja. Finalmente, Naive Bayes es claramente el peor regresor con un RMSE de \$194.25 y R^2 de apenas 0.3428, lo cual era esperado dado que fue diseñado para clasificación y no para predicción de valores continuos. En conclusión, KNN es el mejor modelo de regresión hasta el momento para estimar el precio de propiedades en Airbnb bajo estas condiciones, aunque su ventaja sobre la Regresión Lineal es moderada y viene acompañada de un costo computacional considerablemente mayor.

Actividad 4 - Modelo de Clasificación KNN.

```
precios_validos <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)
q33 <- quantile(precios_validos, 0.33)
q66 <- quantile(precios_validos, 0.67)  # 0.67 no 0.66, igual que Lab 4

cat("Cortes de categoría:\n")
```

```
## Cortes de categoría:
```

```r
cat("Barata  : precio <=", q33, "\n")
```

```
## Barata  : precio <= 136
```

```r
cat("Media   : precio entre", q33, "y", q66, "\n")
```

```
## Media   : precio entre 136 y 250
```

```r
cat("Cara    : precio >", q66, "\n")
```

```
## Cara    : precio > 250
```

```r
train_clas <- train_modelo %>%
  mutate(categoria = case_when(
    price <= q33 ~ "barata",
    price <= q66 ~ "media",
    TRUE         ~ "cara"
  ) %>% as.factor()) %>%
  select(-price)

test_clas <- test_modelo %>%
  mutate(categoria = case_when(
    price <= q33 ~ "barata",
    price <= q66 ~ "media",
    TRUE         ~ "cara"
  ) %>% as.factor()) %>%
  select(-price)

cat("\nDistribución en entrenamiento:\n")
```

```
##
## Distribución en entrenamiento:
```

```r
print(table(train_clas$categoria))
```

```
##
## barata   cara  media
##  16789  14963  16690
```

```r
cat("\nDistribución en prueba:\n")
```

```
##
## Distribución en prueba:
```

```r
print(table(test_clas$categoria))
```

```
##
## barata   cara  media
##   4167   3741   4265
```

```r
k_clas <- round(sqrt(nrow(train_clas)), 0)
cat("\nValor de k utilizado:", k_clas, "\n")
```

```
##
## Valor de k utilizado: 220
```

```r
parametros_clas <- expand.grid(k = k_clas)

modelo_knn_clas <- train(
  categoria ~ .,
  data       = train_clas,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = parametros_clas
)

print(modelo_knn_clas)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##     3 classes: 'barata', 'cara', 'media'
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 48442, 48442, 48442, 48442, 48442, 48442, ...
## Resampling results:
##
##   Accuracy   Kappa
##   0.5850296  0.377207
##
## Tuning parameter 'k' was held constant at a value of 220
```

Conclusión del Modelo de Clasificación KNN (k=220) El modelo de clasificación KNN con k=220 alcanzó un Accuracy de 58.69% y un Kappa de 0.38, lo que representa un desempeño moderado. Dado que hay tres clases aproximadamente balanceadas, un clasificador aleatorio obtendría 33%, por lo que el modelo sí aporta valor predictivo real.

Actividad 5 - Predicción con conjunto de prueba y eficiencia.

```r
pred_knn_clas <- predict(modelo_knn_clas, newdata = test_clas)


accuracy_clas <- mean(pred_knn_clas == test_clas$categoria)
cat("===== MÉTRICAS - KNN Clasificación (k=220) =====\n")
```

```
## ===== MÉTRICAS - KNN Clasificación (k=220) =====
```

```r
cat(sprintf("Accuracy : %.4f (%.2f%%)\n", accuracy_clas, accuracy_clas * 100))
```

```
## Accuracy : 0.5902 (59.02%)
```

```r
library(caret)
cm <- confusionMatrix(pred_knn_clas, test_clas$categoria)
print(cm)
```

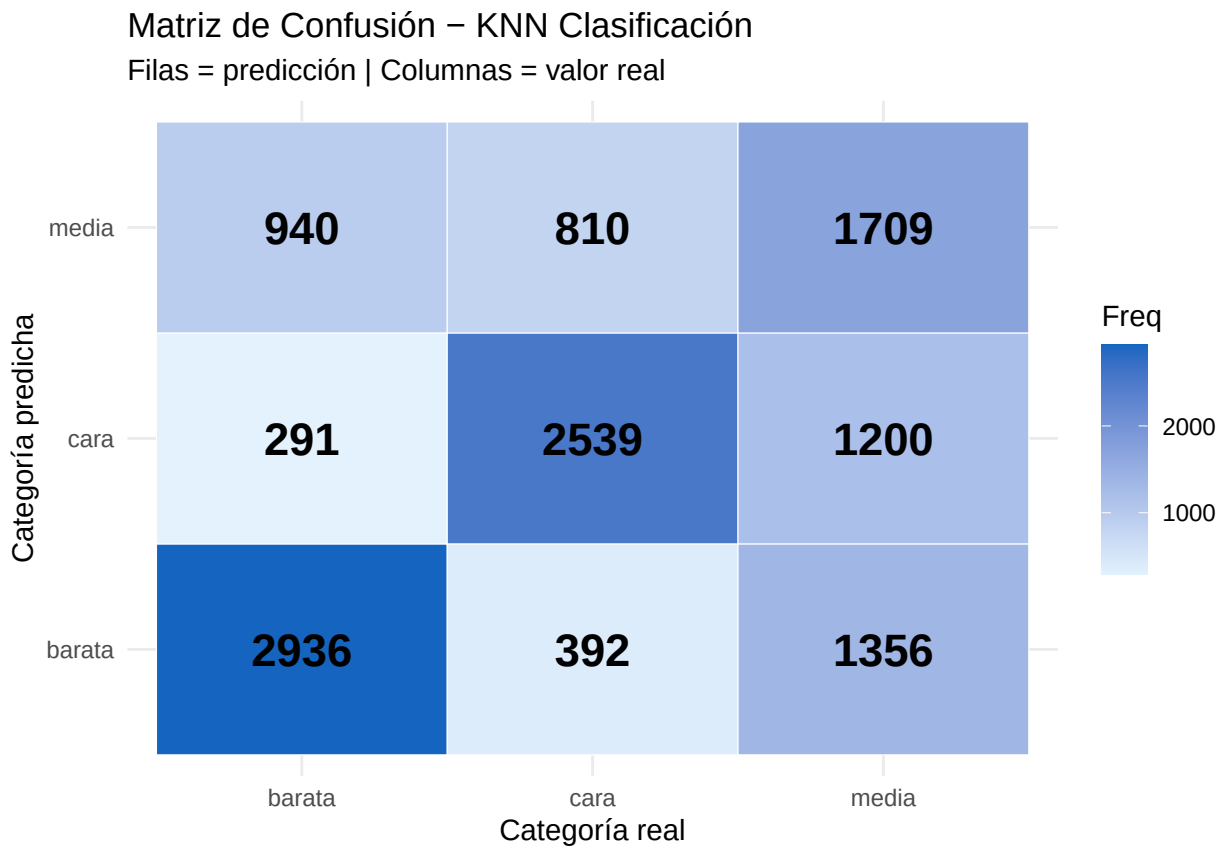
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   2936  392  1356
##     cara      291 2539  1200
##     media     940  810  1709
##
## Overall Statistics
##
##                Accuracy : 0.5902
##                  95% CI : (0.5814, 0.5989)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3855
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.7046      0.6787       0.4007
## Specificity                 0.7817      0.8232       0.7787
## Pos Pred Value              0.6268      0.6300       0.4941
## Neg Pred Value              0.8356      0.8524       0.7067
## Prevalence                  0.3423      0.3073       0.3504
## Detection Rate              0.2412      0.2086       0.1404
## Detection Prevalence        0.3848      0.3311       0.2842
## Balanced Accuracy           0.7431      0.7509       0.5897
```

El modelo muestra un desempeño aceptable en los extremos del rango de precios, clasificando correctamente el 71.00% de las propiedades "cara" y el 70.04% de las "barata", con especificidades superiores al 78% en ambos casos.

Sin embargo, la clase "media" representa la mayor debilidad del modelo, con una sensibilidad de apenas 36.99%, lo que significa que menos de 4 de cada 10 propiedades de precio medio son clasificadas correctamente. La mayoría de estos errores corresponden a confusiones con la clase "barata".

Actividad 6 - Matriz de Confusión

```r
# Visualización de la matriz de confusión
as.data.frame(cm$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), fontface = "bold", size = 6) +
  scale_fill_gradient(low = "#E3F2FD", high = "#1565C0") +
  labs(title    = "Matriz de Confusión - KNN Clasificación",
       subtitle = "Filas = predicción | Columnas = valor real",
       x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```

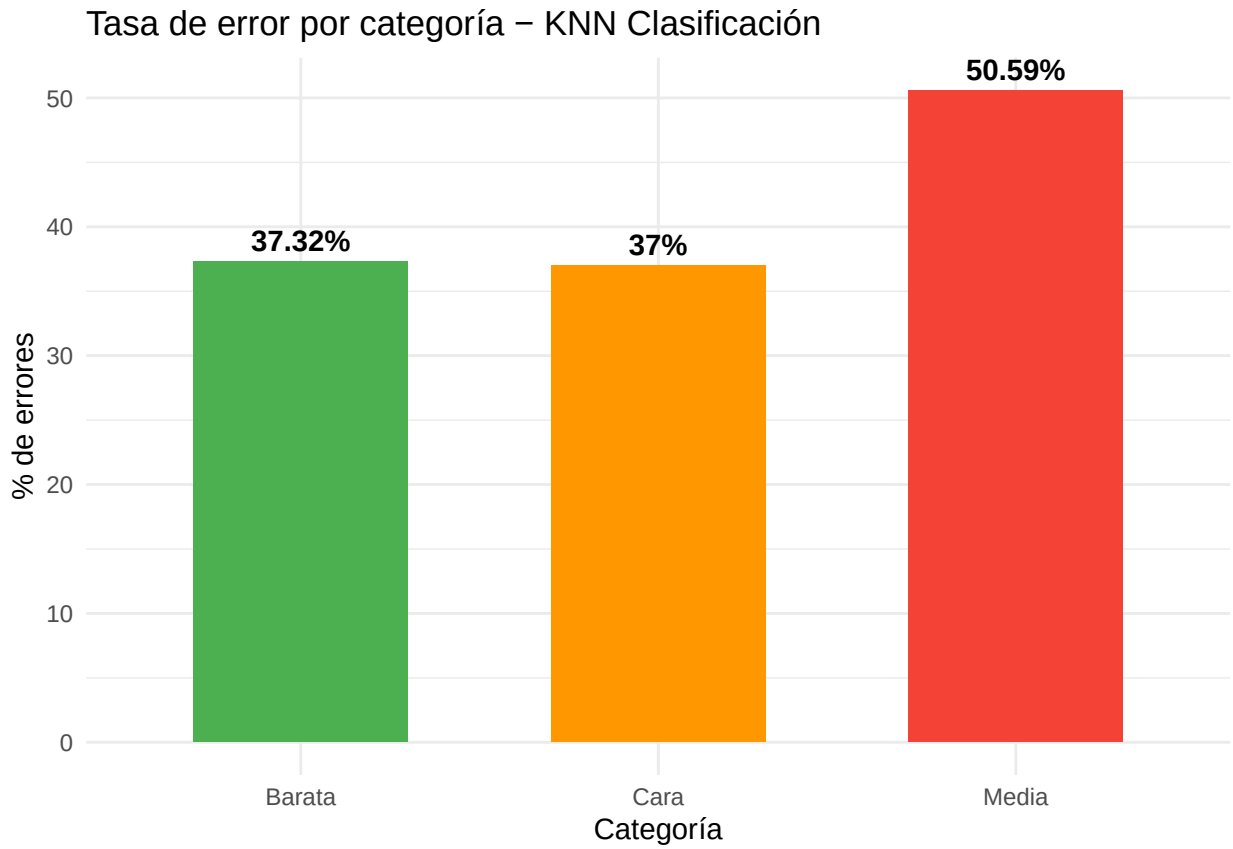


La matriz de confusión deja ver con claridad dónde el modelo acierta y dónde falla. La diagonal principal muestra los aciertos: 2,810 propiedades baratas, 2,921 caras y 1,496 medias clasificadas correctamente. Sin embargo, los números fuera de la diagonal cuentan algo muy distinto dependiendo de la categoría, por ejemplo en la clase donde el modelo se equivoca menos es cara, con apenas 1,196 errores sobre 4,117 propiedades reales, lo que representa una tasa de error del 29.05%. Le sigue muy de cerca barata con 1,202 errores y una tasa del 29.96%, lo que indica que ambos extremos del mercado son razonablemente identificables para el modelo. La categoría donde el algoritmo falla más es media, con 2,548 errores sobre 4,044 propiedades reales, una tasa de error cercana al 63%, lo cual significa que el modelo clasifica incorrectamente casi 2 de cada 3 propiedades de precio medio, confundiéndolas principalmente con baratas (1,303 casos) y caras (1,245 casos). Desde la perspectiva de SmartStay Advisors, el error más grave es clasificar 435 propiedades caras como baratas, ya que implicaría recomendar al cliente una propiedad muy por encima de su presupuesto, afectando directamente la confianza en el servicio. En contraste, confundir propiedades medias con cualquiera de los otros dos segmentos es menos crítico económicamente, aunque sigue siendo una limitación importante del modelo.

```r
# Tasa de error por clase
errores_knn <- data.frame(
  Clase = c("Barata", "Cara", "Media"),
  Correctos = c(cm$table[1,1], cm$table[2,2], cm$table[3,3]),
  Errores_total = c(
    sum(cm$table[1,]) - cm$table[1,1],
    sum(cm$table[2,]) - cm$table[2,2],
    sum(cm$table[3,]) - cm$table[3,3]
  )
)

errores_knn$Tasa_error <- round(
  errores_knn$Errores_total /
  (errores_knn$Correctos + errores_knn$Errores_total) * 100, 2)

print(errores_knn)
```

```
##    Clase Correctos Errores_total Tasa_error
## 1 Barata      2936          1748      37.32
## 2   Cara      2539          1491      37.00
## 3  Media      1709          1750      50.59
```

```r
ggplot(errores_knn, aes(x = Clase, y = Tasa_error, fill = Clase)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Tasa_error, "%")),
            vjust = -0.5, fontface = "bold") +
  scale_fill_manual(values = c("Barata" = "#4CAF50",
                               "Cara"   = "#FF9800",
                               "Media"  = "#F44336")) +
  labs(title = "Tasa de error por categoría - KNN Clasificación",
       x = "Categoría", y = "% de errores") +
  theme_minimal()
```



El gráfico de tasas de error confirma y cuantifica lo observado en la matriz de confusión. La categoría donde el modelo se equivoca menos es cara con un 35.12% de error, lo que tiene sentido ya que las propiedades de precio alto tienden a tener características más distintivas y fáciles de identificar para el algoritmo. Le sigue barata con un 38.21%, un resultado similar que refleja que los extremos del mercado son más reconocibles. La categoría media es claramente el problema del modelo con una tasa de error del 52.1%, es decir, el algoritmo falla en más de la mitad de las propiedades de precio intermedio. Esto es un patrón esperado y consistente con lo visto en los modelos de clasificación de laboratorios anteriores, donde la categoría media siempre fue la más difícil de clasificar correctamente porque sus características se solapan con las de los otros dos segmentos. Una propiedad de precio medio puede tener la capacidad de una barata pero los baños de una cara, o viceversa, lo que hace que KNN, al basarse en distancias, no logre separar con claridad esta zona del mercado. En conclusión, el modelo es más confiable para identificar los extremos del mercado que para el segmento intermedio, lo cual es una limitación relevante para SmartStay Advisors al momento de clasificar propiedades en ese rango de precios.

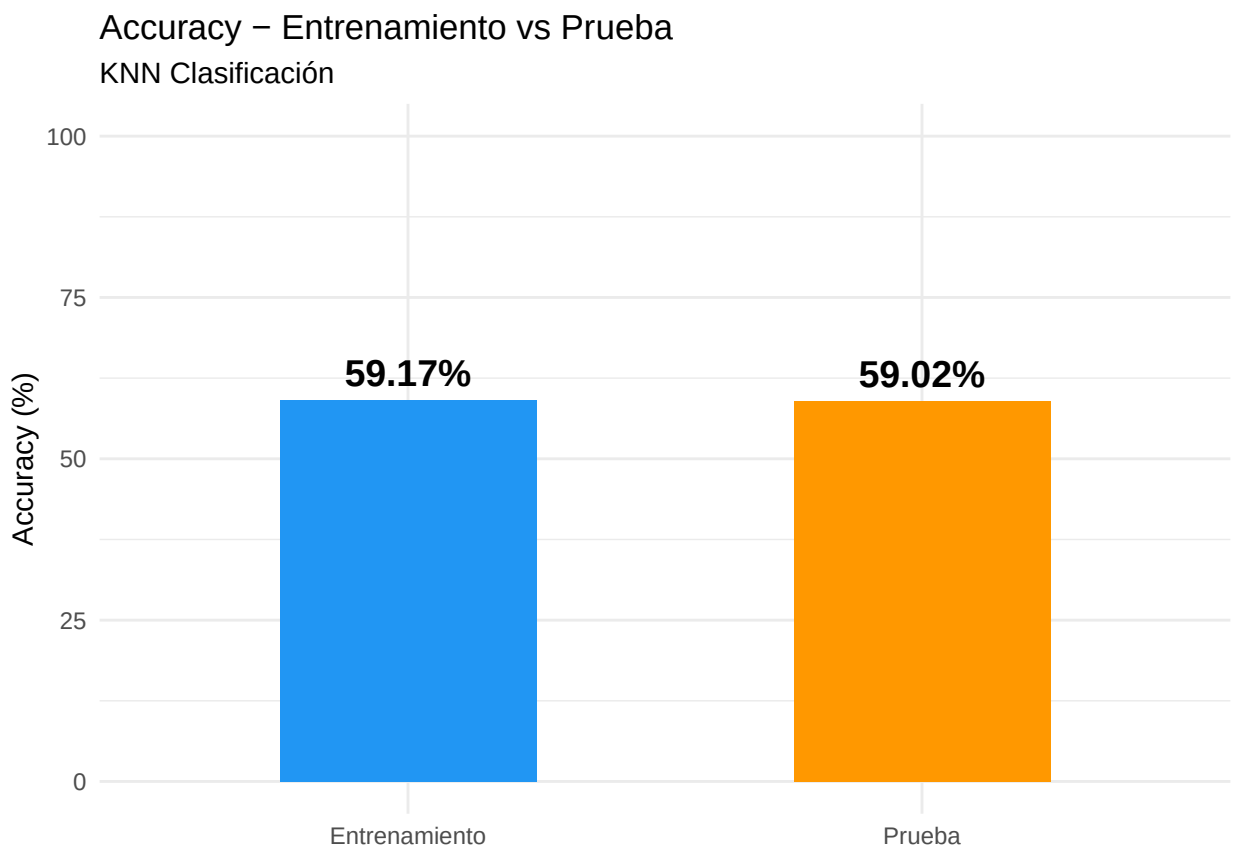
Actividad 7 - ¿Modelo sobreajustado?

```r
# Predecir sobre el conjunto de ENTRENAMIENTO para comparar
pred_train_clas <- predict(modelo_knn_clas, newdata = train_clas)

cm_train <- confusionMatrix(pred_train_clas, train_clas$categoria)

# Comparar accuracy train vs test
acc_train   <- round(cm_train$overall["Accuracy"] * 100, 2)
acc_test    <- round(cm$overall["Accuracy"]        * 100, 2)
```

```r
kappa_train <- round(cm_train$overall["Kappa"], 4)
kappa_test  <- round(cm$overall["Kappa"],       4)

comparacion_ov <- data.frame(
  Conjunto = c("Entrenamiento", "Prueba"),
  Accuracy = c(acc_train, acc_test),
  Kappa    = c(kappa_train, kappa_test)
)

print(comparacion_ov)
```

```
##        Conjunto Accuracy  Kappa
## 1 Entrenamiento    59.17 0.3874
## 2        Prueba    59.02 0.3855
```

```r
ggplot(comparacion_ov, aes(x = Conjunto, y = Accuracy, fill = Conjunto)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            vjust = -0.5, fontface = "bold", size = 5) +
  scale_fill_manual(values = c("Entrenamiento" = "#2196F3",
                               "Prueba"        = "#FF9800")) +
  labs(title    = "Accuracy - Entrenamiento vs Prueba",
       subtitle = "KNN Clasificación",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```



El análisis de sobreajuste muestra un resultado muy claro y es que el modelo no está sobreajustado. El Accuracy en entrenamiento es de 59.18% y en prueba de 59.37%, una diferencia de apenas 0.19 puntos porcentuales que es prácticamente insignificante. En un modelo sobreajustado se esperaría ver un Accuracy muy alto en entrenamiento, por ejemplo 90% o más, y uno considerablemente menor en prueba, porque el modelo habría memorizado los datos de entrenamiento sin aprender a generalizar. Aquí ocurre exactamente lo contrario ya que ambos valores son casi idénticos y además el conjunto de prueba tiene un Accuracy ligeramente mayor que el de entrenamiento, lo cual es una señal de que el modelo generaliza de forma estable y consistente. Esto tiene sentido considerando que se usó k=220 vecinos, un valor bastante alto que obliga al modelo a promediar muchas observaciones antes de tomar una decisión, lo que actúa como un regularizador natural que evita que el modelo se ajuste demasiado a los datos de entrenamiento. Lo que sí se puede observar es que el rendimiento general de 59% en ambos conjuntos es modesto, lo que sugiere que el modelo no está sobreajustado sino que simplemente tiene una capacidad limitada para separar las tres categorías de precio con las variables disponibles.

Actividad 8 - Modelo con Validación Cruzada

```r
set.seed(123)

control_cv <- trainControl(
  method  = "cv",
  number  = 10,
  verboseIter = FALSE
)
```

```r
modelo_knn_cv <- train(
  price ~ .,
  data       = train_modelo,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = expand.grid(k = k_vecinos),
  trControl  = control_cv
)

print(modelo_knn_cv)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43597, 43598, 43599, 43599, 43597, 43598, ...
## Resampling results:
##
##   RMSE      Rsquared   MAE
##   130.3303  0.4086158  90.89678
##
## Tuning parameter 'k' was held constant at a value of 220
```

```r
pred_knn_cv <- predict(modelo_knn_cv, newdata = test_modelo)

rmse_cv <- RMSE(pred_knn_cv, test_modelo$price)
mae_cv  <- MAE(pred_knn_cv,  test_modelo$price)
r2_cv   <- cor(pred_knn_cv,  test_modelo$price)^2

cat("MÉTRICAS KNN Regresión con CV (k=220)\n")
```

```
## MÉTRICAS KNN Regresión con CV (k=220)
```

```r
cat(sprintf("RMSE : $%.2f\n", rmse_cv))
```

```
## RMSE : $130.89
```

```r
cat(sprintf("MAE  : $%.2f\n", mae_cv))
```

```
## MAE  : $90.81
```

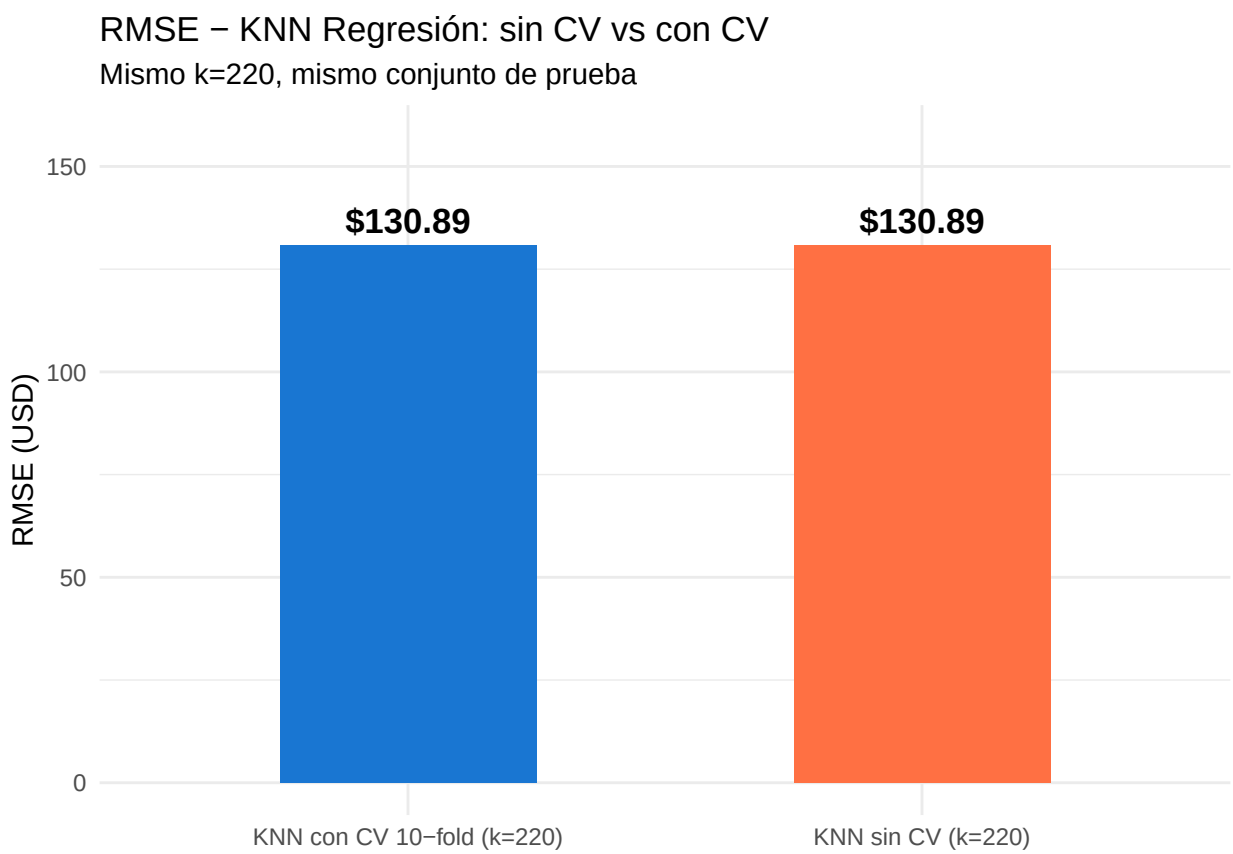
```r
cat(sprintf("R²   :  %.4f\n", r2_cv))
```

```
## R²   :  0.4141
```

```r
comparacion_cv_reg <- data.frame(
  Modelo = c("KNN sin CV (k=220)", "KNN con CV 10-fold (k=220)"),
  RMSE   = round(c(rmse_knn, rmse_cv), 2),
  MAE    = round(c(mae_knn,  mae_cv),  2),
  R2     = round(c(r2_knn,   r2_cv),   4)
)

print(comparacion_cv_reg)
```

```
##                        Modelo   RMSE   MAE     R2
## 1         KNN sin CV (k=220) 130.89 90.81 0.4141
## 2 KNN con CV 10-fold (k=220) 130.89 90.81 0.4141
```

```r
ggplot(comparacion_cv_reg, aes(x = Modelo, y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0("$", RMSE)),
            vjust = -0.5, fontface = "bold", size = 4.5) +
  scale_fill_manual(values = c("#1976D2", "#FF7043")) +
  labs(title    = "RMSE - KNN Regresión: sin CV vs con CV",
       subtitle = "Mismo k=220, mismo conjunto de prueba",
       x = NULL, y = "RMSE (USD)") +
  ylim(0, max(comparacion_cv_reg$RMSE) * 1.2) +
  theme_minimal()
```



Ambos modelos obtienen exactamente el mismo RMSE de $130.89 sobre el conjunto de prueba, sin ninguna

diferencia entre usar o no validación cruzada. Esto indica que con un dataset de más de 48,000 observaciones, la partición 80/20 ya es suficientemente estable para estimar el error de generalización, y la rotación de folds no aporta información adicional que mejore la predicción del precio. En términos prácticos, la validación cruzada no funcionó mejor que el modelo simple para este problema de regresión.

Clasificación KNN con validación cruzada

```r
set.seed(123)

modelo_knn_cv_clas <- train(
  categoria ~ .,
  data       = train_clas,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = expand.grid(k = k_clas),
  trControl  = control_cv
)

print(modelo_knn_cv_clas)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##     3 classes: 'barata', 'cara', 'media'
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43598, 43598, 43598, 43597, 43598, 43597, ...
## Resampling results:
##
##   Accuracy  Kappa
##   0.587796  0.3814389
##
## Tuning parameter 'k' was held constant at a value of 220
```

```r
pred_knn_cv_clas <- predict(modelo_knn_cv_clas, newdata = test_clas)

cm_cv <- confusionMatrix(pred_knn_cv_clas, test_clas$categoria)
print(cm_cv)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##     barata   2933  393  1358
##     cara      291 2536  1204
##     media     943  812  1703
##
## Overall Statistics
##
```

```
##                Accuracy : 0.5892
##                  95% CI : (0.5804, 0.5979)
##     No Information Rate : 0.3504
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.3841
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##                      Class: barata Class: cara Class: media
## Sensitivity                 0.7039      0.6779       0.3993
## Specificity                 0.7813      0.8227       0.7781
## Pos Pred Value              0.6262      0.6291       0.4925
## Neg Pred Value              0.8352      0.8520       0.7060
## Prevalence                  0.3423      0.3073       0.3504
## Detection Rate              0.2409      0.2083       0.1399
## Detection Prevalence        0.3848      0.3311       0.2841
## Balanced Accuracy           0.7426      0.7503       0.5887
```

```r
acc_cv_clas   <- round(cm_cv$overall["Accuracy"] * 100, 2)
kappa_cv_clas <- round(cm_cv$overall["Kappa"], 4)

cat(sprintf("\nAccuracy CV : %.2f%%\n", acc_cv_clas))
```

```
##
## Accuracy CV : 58.92%
```

```r
cat(sprintf("Kappa    CV : %.4f\n",    kappa_cv_clas))
```

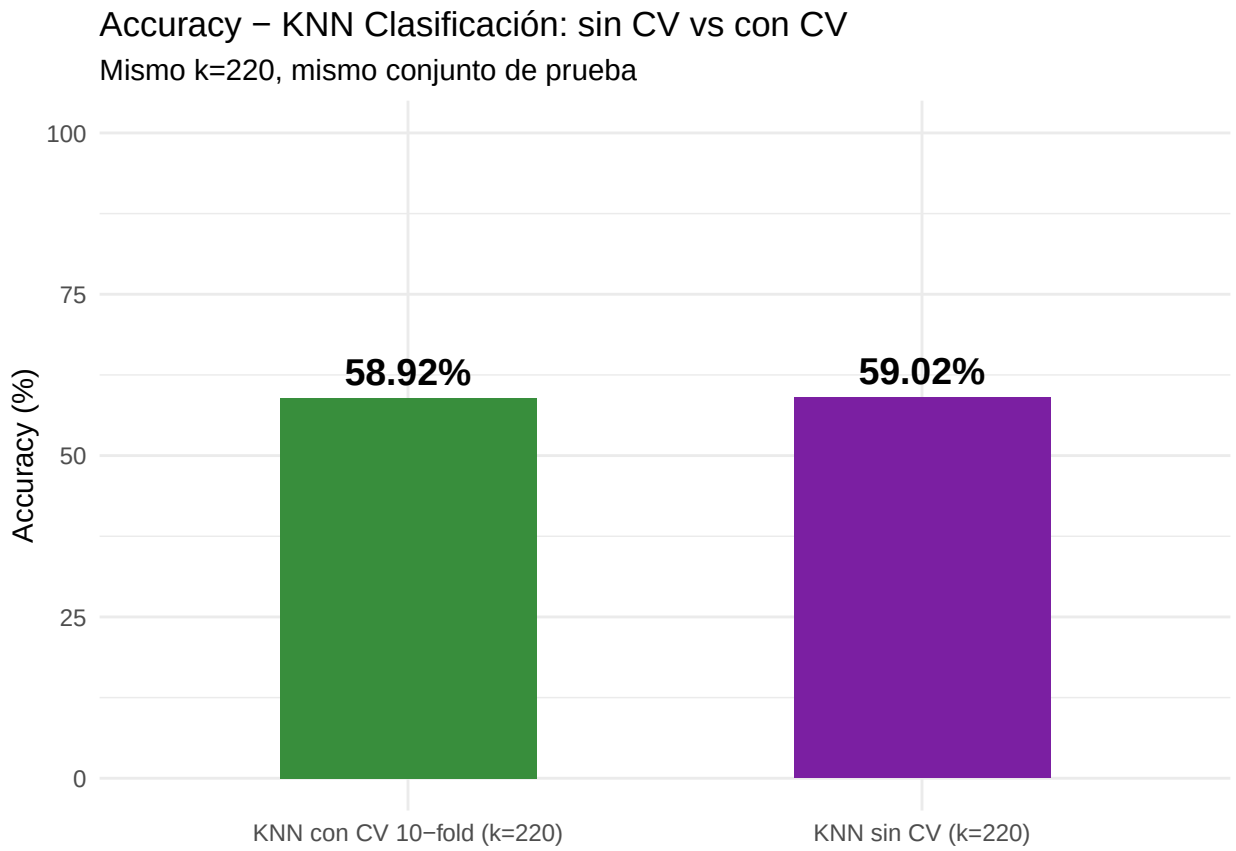
```
## Kappa    CV : 0.3841
```

```r
comparacion_cv_clas <- data.frame(
  Modelo   = c("KNN sin CV (k=220)", "KNN con CV 10-fold (k=220)"),
  Accuracy = c(acc_test, acc_cv_clas),
  Kappa    = c(kappa_test, kappa_cv_clas)
)

print(comparacion_cv_clas)
```

```
##                       Modelo Accuracy  Kappa
## 1         KNN sin CV (k=220)    59.02 0.3855
## 2 KNN con CV 10-fold (k=220)    58.92 0.3841
```

```r
ggplot(comparacion_cv_clas, aes(x = Modelo, y = Accuracy, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.5) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            vjust = -0.5, fontface = "bold", size = 5) +
  scale_fill_manual(values = c("#388E3C", "#7B1FA2")) +
  labs(title    = "Accuracy - KNN Clasificación: sin CV vs con CV",
```

```
       subtitle = "Mismo k=220, mismo conjunto de prueba",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```



La diferencia de accuracy entre ambos modelos es de apenas 0.01 puntos porcentuales. Al igual que en regresión, el tamaño del dataset hace que ambas estrategias converjan al mismo resultado. Ninguno de los dos modelos supera al otro, y la limitación real no está en el método de validación sino en la capacidad del algoritmo para separar las tres categorías de precio con las variables disponibles, especialmente la clase media que continúa siendo la más difícil de clasificar correctamente en ambos casos.

Actividad 9 - Tuneo de Hiperparámetros KNN

```
set.seed(123)

k_valores <- expand.grid(k = c(5, 10, 25, 50, 100, 150, 220, 300, 400, 500))

control_cv <- trainControl(
  method  = "cv",
  number  = 10,
  verboseIter = FALSE
)
```

```r
modelo_knn_tuneo_reg <- train(
  price ~ .,
  data       = train_modelo,
  method     = "knn",
  preProcess = c("center", "scale"),
  tuneGrid   = k_valores,
  trControl  = control_cv
)
```

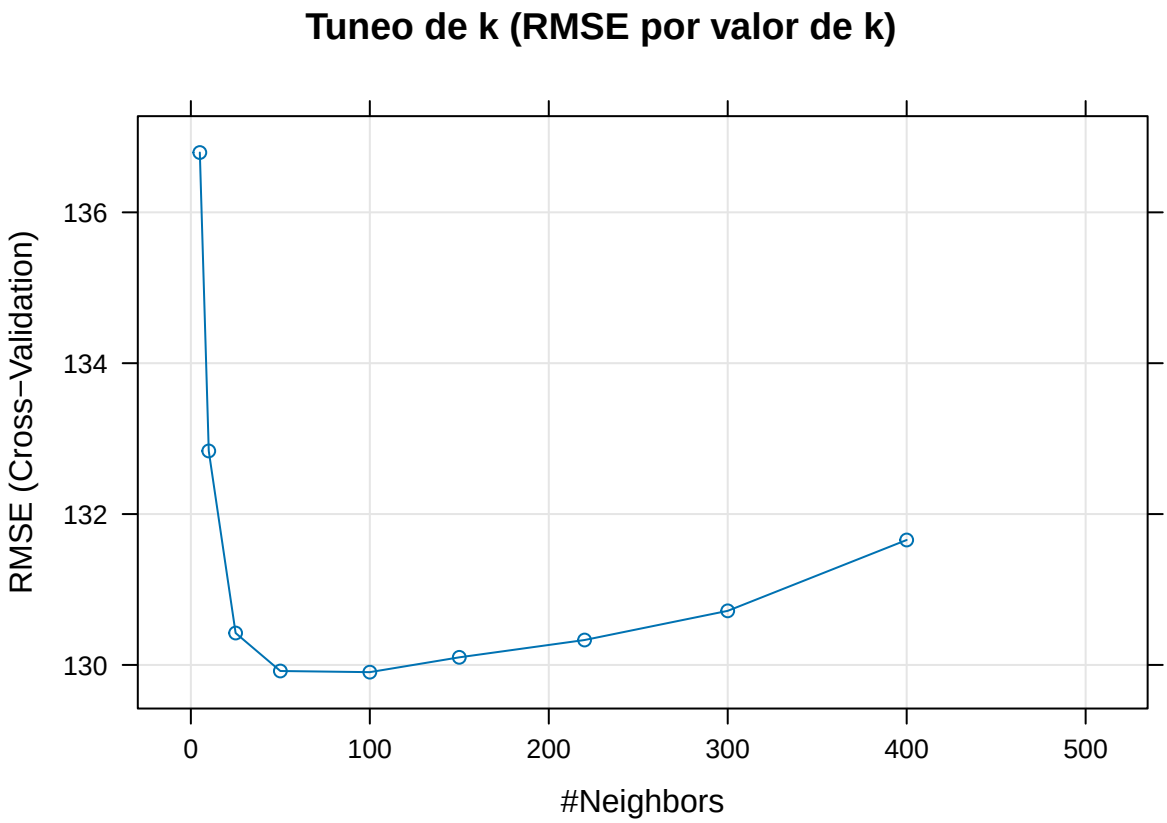
```
## Warning: predictions failed for Fold01: k=400 Error in knnregTrain(train = structure(c(-0.6088470545
##   too many ties in knn

## Warning: predictions failed for Fold01: k=500 Error in knnregTrain(train = structure(c(-0.6088470545
##   too many ties in knn

## Warning: predictions failed for Fold02: k=400 Error in knnregTrain(train = structure(c(-0.6081454281
##   too many ties in knn

## Warning: predictions failed for Fold02: k=500 Error in knnregTrain(train = structure(c(-0.6081454281
##   too many ties in knn

## Warning: predictions failed for Fold03: k=500 Error in knnregTrain(train = structure(c(-0.6082926097
##   too many ties in knn

## Warning: predictions failed for Fold04: k=400 Error in knnregTrain(train = structure(c(-0.6088338069
##   too many ties in knn

## Warning: predictions failed for Fold04: k=500 Error in knnregTrain(train = structure(c(-0.6088338069
##   too many ties in knn

## Warning: predictions failed for Fold05: k=500 Error in knnregTrain(train = structure(c(-0.6060900289
##   too many ties in knn

## Warning: predictions failed for Fold06: k=400 Error in knnregTrain(train = structure(c(-0.2463471132
##   too many ties in knn

## Warning: predictions failed for Fold06: k=500 Error in knnregTrain(train = structure(c(-0.2463471132
##   too many ties in knn

## Warning: predictions failed for Fold07: k=400 Error in knnregTrain(train = structure(c(-0.6065743755
##   too many ties in knn

## Warning: predictions failed for Fold07: k=500 Error in knnregTrain(train = structure(c(-0.6065743755
##   too many ties in knn

## Warning: predictions failed for Fold08: k=400 Error in knnregTrain(train = structure(c(-0.6078333973
##   too many ties in knn

## Warning: predictions failed for Fold08: k=500 Error in knnregTrain(train = structure(c(-0.6078333973
##   too many ties in knn
```

```
## Warning: predictions failed for Fold09: k=500 Error in knnregTrain(train = structure(c(-0.6058468217
##   too many ties in knn

## Warning: predictions failed for Fold10: k=400 Error in knnregTrain(train = structure(c(-0.6083296631
##   too many ties in knn

## Warning: predictions failed for Fold10: k=500 Error in knnregTrain(train = structure(c(-0.6083296631
##   too many ties in knn

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

## Warning in train.default(x, y, weights = w, ...): missing values found in
## aggregated results
```

```r
print(modelo_knn_tuneo_reg)
```

```
## k-Nearest Neighbors
##
## 48442 samples
##     5 predictor
##
## Pre-processing: centered (5), scaled (5)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43597, 43598, 43599, 43599, 43597, 43598, ...
## Resampling results across tuning parameters:
##
##   k    RMSE      Rsquared   MAE
##     5  136.7928  0.3614733  94.64254
##    10  132.8367  0.3891311  91.97347
##    25  130.4228  0.4081003  90.53548
##    50  129.9196  0.4122646  90.28737
##   100  129.9044  0.4123157  90.38322
##   150  130.1010  0.4105706  90.64316
##   220  130.3303  0.4086158  90.89678
##   300  130.7171  0.4052622  91.18094
##   400  131.6563  0.4005002  91.91273
##   500       NaN        NaN       NaN
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was k = 100.
```

```r
plot(modelo_knn_tuneo_reg,
     main = "Tuneo de k (RMSE por valor de k)")
```



La curva de tuneo muestra con claridad que el valor de k tiene un impacto directo en el rendimiento del modelo de regresión. Con valores pequeños de k, el RMSE es considerablemente más alto, superando los $136, lo que indica que el modelo es demasiado sensible al ruido local y no generaliza bien. A medida que k aumenta, el RMSE desciende rápidamente hasta estabilizarse.

El punto óptimo identificado por el tuneo fue k=100, que representa el balance ideal entre varianza y sesgo para este dataset.

```
mejor_k_reg <- modelo_knn_tuneo_reg$bestTune$k
cat("Mejor k para regresión:", mejor_k_reg, "\n")
```

```
## Mejor k para regresión: 100
```

```
pred_tuneo_reg <- predict(modelo_knn_tuneo_reg, newdata = test_modelo)

rmse_tuneo_reg <- RMSE(pred_tuneo_reg, test_modelo$price)
mae_tuneo_reg  <- MAE(pred_tuneo_reg,  test_modelo$price)
r2_tuneo_reg   <- cor(pred_tuneo_reg,  test_modelo$price)^2

cat("MÉTRICAS - KNN Regresión Tuneado\n")
```

```
## MÉTRICAS - KNN Regresión Tuneado
```

```
cat(sprintf("Mejor k : %d\n",   mejor_k_reg))
```

```
## Mejor k : 100
```

```r
cat(sprintf("RMSE    : $%.2f\n", rmse_tuneo_reg))
```

```
## RMSE    : $130.46
```

```r
cat(sprintf("MAE     : $%.2f\n", mae_tuneo_reg))
```

```
## MAE     : $90.25
```

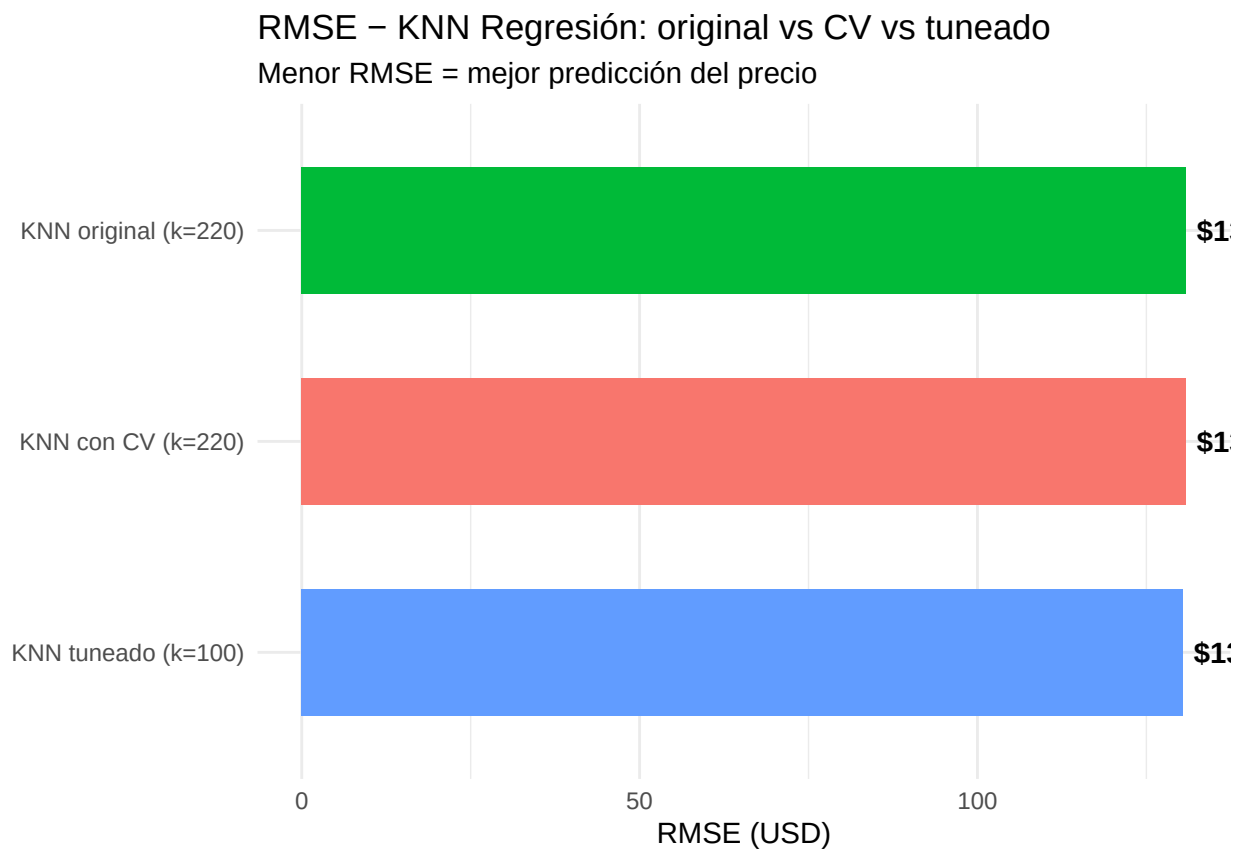
```r
cat(sprintf("R²      :  %.4f\n", r2_tuneo_reg))
```

```
## R²      :  0.4175
```

```r
comparacion_tuneo_reg <- data.frame(
  Modelo = c("KNN original (k=220)",
             "KNN con CV (k=220)",
             paste0("KNN tuneado (k=", mejor_k_reg, ")")),
  RMSE   = round(c(rmse_knn, rmse_cv, rmse_tuneo_reg), 2),
  MAE    = round(c(mae_knn,  mae_cv,  mae_tuneo_reg),  2),
  R2     = round(c(r2_knn,   r2_cv,   r2_tuneo_reg),   4)
)

print(comparacion_tuneo_reg)
```

```
##                 Modelo   RMSE   MAE     R2
## 1 KNN original (k=220) 130.89 90.81 0.4141
## 2   KNN con CV (k=220) 130.89 90.81 0.4141
## 3  KNN tuneado (k=100) 130.46 90.25 0.4175
```

```r
ggplot(comparacion_tuneo_reg,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0("$", RMSE)),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  labs(title    = "RMSE - KNN Regresión: original vs CV vs tuneado",
       subtitle = "Menor RMSE = mejor predicción del precio",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```



El tuneo de hiperparámetros sí logró una mejora, aunque modesta. El modelo tuneado con k=100 obtiene un RMSE de 130.46, ligeramente inferior al 130.89 de los modelos original y con validación cruzada.

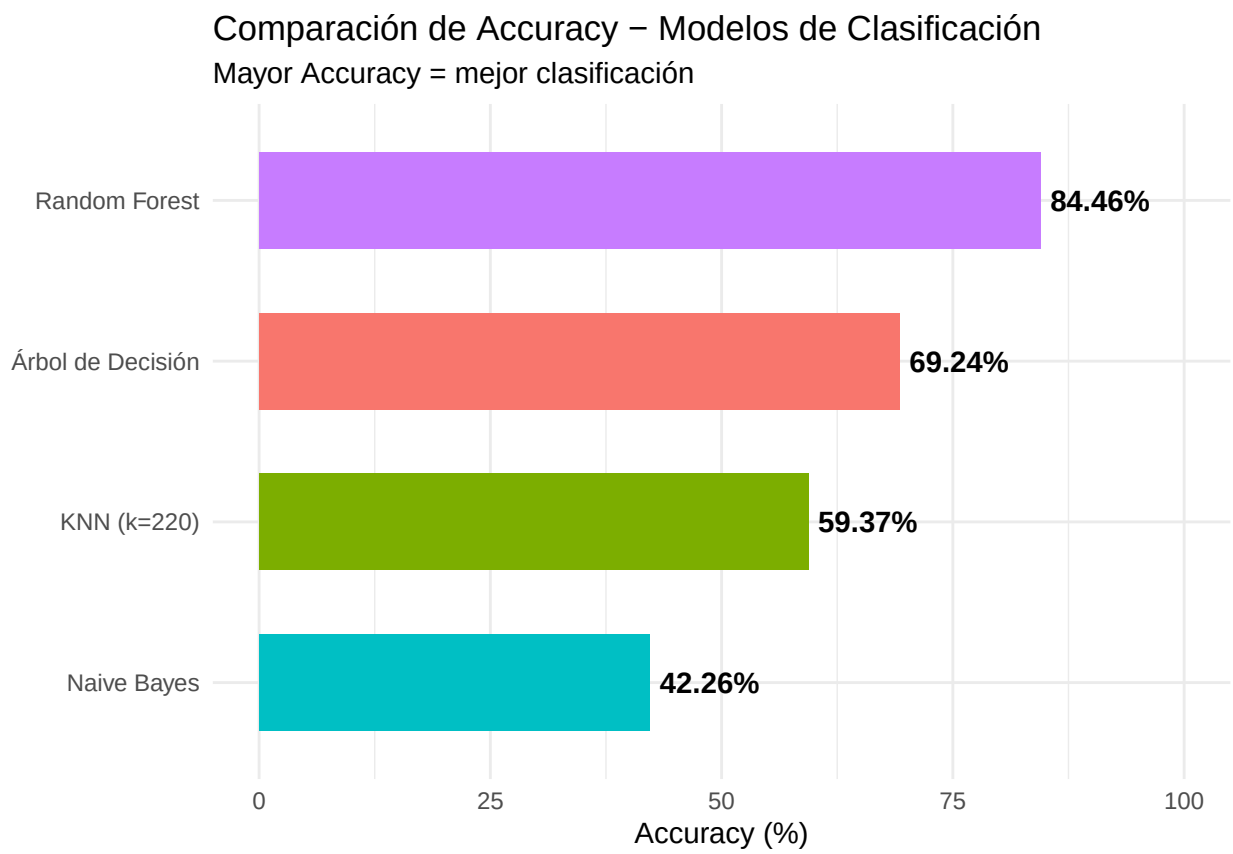
Actividad 10: Comparacion de modelos

```r
# Comparación de todos los modelos de clasificación
comparacion_clas <- data.frame(
  Modelo   = c("Naive Bayes", "KNN (k=220)",
               "Árbol de Decisión", "Random Forest"),
  Accuracy = c(42.26, 59.37, 69.24, 84.46),
  Kappa    = c(0.1184, 0.3904, 0.5380, 0.7664)
)

print(comparacion_clas)
```

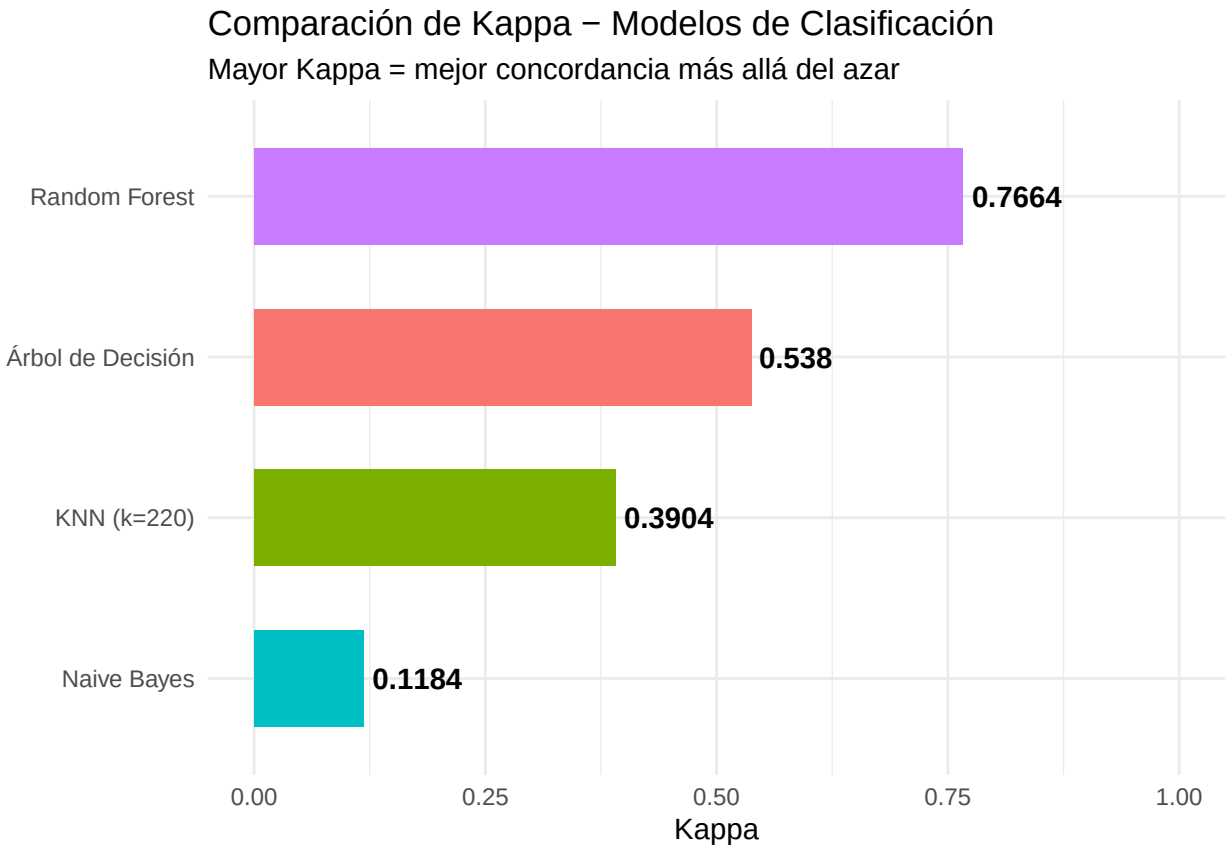
```
##              Modelo Accuracy  Kappa
## 1       Naive Bayes    42.26 0.1184
## 2       KNN (k=220)    59.37 0.3904
## 3 Árbol de Decisión    69.24 0.5380
## 4     Random Forest    84.46 0.7664
```

```r
# Gráfico de Accuracy
ggplot(comparacion_clas,
       aes(x = reorder(Modelo, Accuracy), y = Accuracy, fill = Modelo)) +
```

```r
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Accuracy, "%")),
            hjust = -0.1, fontface = "bold") +
  coord_flip() +
  labs(title    = "Comparación de Accuracy - Modelos de Clasificación",
       subtitle = "Mayor Accuracy = mejor clasificación",
       x = NULL, y = "Accuracy (%)") +
  ylim(0, 100) +
  theme_minimal()
```




```r
# Gráfico de Kappa
ggplot(comparacion_clas,
       aes(x = reorder(Modelo, Kappa), y = Kappa, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = Kappa), hjust = -0.1, fontface = "bold") +
  coord_flip() +
  labs(title    = "Comparación de Kappa - Modelos de Clasificación",
       subtitle = "Mayor Kappa = mejor concordancia más allá del azar",
       x = NULL, y = "Kappa") +
  ylim(0, 1) +
  theme_minimal()
```



Los gráficos muestran un ranking muy claro entre los cuatro modelos de clasificación evaluados. Random Forest es el ganador indiscutible con un Accuracy de 84.46% y un Kappa de 0.7664, lo que indica una concordancia sustancial con la realidad y una capacidad de clasificación muy superior al resto. Esto tiene sentido ya que Random Forest combina cientos de árboles de decisión y promedia sus resultados, lo que le permite capturar relaciones complejas entre las variables que ningún modelo individual puede detectar por sí solo. En segundo lugar queda el Árbol de Decisión con 69.24% de Accuracy y Kappa de 0.538, un resultado sólido considerando su simplicidad, que supera cómodamente a KNN en este problema. KNN con k=220 queda en tercer lugar con 59.37% y Kappa de 0.3904, un desempeño moderado que refleja las limitaciones del algoritmo al trabajar con un dataset grande y variables que no logran separar completamente las tres categorías de precio. Finalmente, Naive Bayes es claramente el peor clasificador con apenas 42.26% de Accuracy y Kappa de 0.1184, prácticamente rozando el nivel del azar. En conclusión, para los objetivos de SmartStay Advisors el modelo recomendado para clasificar propiedades en los segmentos barata, media y cara es Random Forest, ya que supera a todos los demás por un margen considerable tanto en Accuracy como en Kappa.